



Modern Academy

Faculty of Engineering
Department of Computer Engineering

NAJDA

Network for AI-powered Joint Dispatch and Assistance

A Graduation Project Submitted in Partial Fulfillment of the
Requirements for the Degree of Bachelor of Engineering in Computer Engineering

Prepared By

Abdelrahman Zaher
Boody
Razan Ihab
Shahd Ahmed
Saif-Allah Mohammed
Yara Emad
Youssef Mohammed
Youssef Ramadan

Supervised By

Dr. Sherin Omran

January, 2027

Approval Page

This graduation project, titled *RapidAid — An Intelligent Emergency Response Simulation Platform*, has been approved by the examination committee as fulfilling the requirements for the degree of Bachelor of Engineering in Computer Engineering.

Supervisor: _____

Date: _____

Committee Member: _____

Date: _____

Committee Member: _____

Date: _____

Department Head: _____

Date: _____

We hereby declare that this graduation project is our own original work, carried out under the supervision of Dr. Sherin Omran, and that it has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution. All sources of information have been duly acknowledged.

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Dr. Sherin Omran, for their continuous guidance, patience, and valuable feedback throughout the development of this project.

We also extend our thanks to the faculty and staff of the Department of Computer Engineering for providing the resources and support necessary to complete this work, and to our families and friends for their encouragement throughout this journey.

Emergency response coordination requires rapid decision-making across multiple agencies — ambulance, fire, and police services — under significant time pressure. Existing student and research projects in this space often either oversimplify the problem into a basic request-tracking application, or attempt to replicate real national emergency systems, introducing legal and operational constraints that make them undemonstrable in an academic setting.

This project presents NAJDA (Network for AI-powered Joint Dispatch and Assistance), a simulation and decision-support platform that models intelligent emergency dispatch workflows for educational and research purposes. The system is built as an event-driven, microservice-based architecture comprising a citizen-facing mobile application, role-based responder mobile applications, a real-time dispatcher web dashboard, a hospital coordination interface, and an administrative panel. The platform combines custom-built domain services with managed cloud infrastructure — a hosted PostgreSQL database as the relational system of record, and Firebase for authentication, file storage, and push notifications — allowing engineering effort to be concentrated on the system’s differentiating logic rather than commodity infrastructure.

An artificial intelligence service assists — but does not replace — human decision-makers by predicting incident priority, flagging duplicate reports, and recommending optimal responder units and hospitals based on proximity, availability, and capacity. The platform demonstrates real-time communication through WebSocket-based live tracking, asynchronous service coordination through a message broker, and machine learning techniques including gradient-boosted classification and image-based hazard detection.

Once evaluation is complete, summarize the key measured outcome here – e.g., classifier accuracy on the test set, or simulated average dispatch decision time.

This work demonstrates the feasibility of applying modern distributed systems and applied artificial intelligence techniques to a socially relevant problem domain within the scope of a Computer Engineering graduation project.

Keywords: Emergency Dispatch, Event-Driven Architecture, Real-Time Systems, Microservices, Machine Learning, Decision Support Systems, Cloud Infrastructure

Contents

Approval Page	i
Declaration	ii
Acknowledgments	iii
Abstract	iv
List of Abbreviations	xiii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Proposed Solution	2
1.4 Project Objectives	3
1.5 Project Scope	3
1.6 Research Methodology Overview	4
1.7 Report Organization	4
2 Requirements Analysis	5
2.1 Introduction	5
2.2 Stakeholders	5
2.2.1 Stakeholder Responsibility Matrix	6
2.3 System Actors	6
2.4 Functional Requirements	6
2.4.1 Citizen Functions	6
2.4.2 Dispatcher Functions	7
2.4.3 Responder Functions	7
2.4.4 Hospital Functions	7
2.4.5 Administrator Functions	7
2.4.6 Functional Requirements Matrix	7
2.5 Non-Functional Requirements	7

2.5.1	Performance	7
2.5.2	Scalability	7
2.5.3	Availability	7
2.5.4	Maintainability	7
2.5.5	Usability	7
2.5.6	Security	8
2.5.7	Non-Functional Requirements Matrix	8
2.6	User Stories	8
2.7	Use Case Analysis	8
2.7.1	Overall Use Case Diagram	8
2.7.2	Use Case Specifications	8
2.8	Business Rules	8
2.9	Assumptions and Constraints	9
2.9.1	Assumptions	9
2.9.2	Constraints	9
2.10	Overall System Workflow	9
2.10.1	Workflow Activity Diagram	10
2.11	Requirements Traceability Matrix	10
2.12	Chapter Summary	10
3	Literature Review	11
3.1	Introduction	11
3.2	Introduction	11
3.3	Emergency Dispatch Systems	11
3.4	Artificial Intelligence in Emergency Response	12
3.5	Microservice Architecture	12
3.6	Event-Driven Systems	13
3.7	Real-Time Communication Technologies	13
3.8	Cloud Services and Backend Technologies	13
3.9	Related Work	14
3.10	Research Gap	14
3.11	Chapter Summary	15
4	System Design	16
4.1	Introduction	16
4.2	Design Goals	16
4.3	Overall System Architecture	17
4.3.1	Overall Architecture Diagram	17
4.4	Architectural Views	17

4.4.1	System Context View	17
4.4.2	Container View	17
4.4.3	Component View	17
4.5	Microservice Architecture	17
4.5.1	Service Responsibilities	17
4.5.2	Inter-Service Communication	17
4.6	Database Design	18
4.6.1	Entity Relationship Diagram	18
4.6.2	Database Schema	18
4.7	Class Design	18
4.7.1	Class Diagram	18
4.8	System Behaviour	18
4.8.1	Incident Reporting Sequence	18
4.8.2	Mission Assignment Sequence	18
4.8.3	Hospital Assignment Sequence	18
4.9	Incident State Model	19
4.9.1	State Diagram	19
4.10	Artificial Intelligence Design	19
4.10.1	AI Architecture	19
4.11	Authentication and Authorization	19
4.11.1	Authentication Flow	19
4.11.2	Authorization Model	20
4.12	Real-Time Communication	20
4.12.1	Real-Time Event Flow	20
4.13	Deployment Architecture	20
4.13.1	Deployment Diagram	20
4.14	Security Considerations	20
4.14.1	Security Architecture	20
4.15	Design Decisions	21
4.16	Chapter Summary	21
5	Implementation	22
5.1	Introduction	22
5.2	Development Environment	22
5.2.1	Software Tools	22
5.2.2	Technology Stack	23
5.3	Backend Implementation	23
5.3.1	Microservice Implementation	23
5.3.2	REST API Implementation	23

5.3.3	Event Processing	23
5.4	Database Implementation	23
5.4.1	Database Tables	23
5.4.2	Relationships	23
5.4.3	Migration Strategy	24
5.5	Mobile Application Implementation	24
5.5.1	Application Structure	24
5.5.2	Citizen Features	24
5.5.3	Responder Features	24
5.6	Web Application Implementation	24
5.6.1	Dispatcher Dashboard	24
5.6.2	Hospital Dashboard	24
5.6.3	Administration Dashboard	24
5.7	Artificial Intelligence Implementation	25
5.7.1	Priority Prediction	25
5.7.2	Duplicate Detection	25
5.7.3	Hospital Recommendation	25
5.7.4	Hazard Classification	25
5.8	Cloud Services Integration	25
5.8.1	Authentication	25
5.8.2	Push Notifications	25
5.8.3	File Storage	25
5.9	Caching Implementation	26
5.10	Security Implementation	26
5.10.1	Authentication	26
5.10.2	Authorization	26
5.10.3	Secure Communication	26
5.11	Deployment	26
5.11.1	Production Environment	27
5.11.2	Deployment Pipeline	27
5.12	Challenges Encountered	27
5.13	Chapter Summary	27
6	Testing and Evaluation	28
6.1	Introduction	28
6.2	Testing Strategy	28
6.3	Testing Environment	29
6.3.1	Hardware Environment	29
6.3.2	Software Environment	29

6.4	Unit Testing	29
6.4.1	Backend Unit Tests	29
6.4.2	Frontend Unit Tests	29
6.4.3	AI Module Tests	29
6.5	Integration Testing	29
6.5.1	Backend Integration	29
6.5.2	Event Processing	30
6.5.3	Cloud Service Integration	30
6.6	System Testing	30
6.6.1	Emergency Reporting	30
6.6.2	Mission Assignment	30
6.6.3	Hospital Recommendation	30
6.6.4	Incident Completion	30
6.7	User Acceptance Testing	30
6.8	Performance Evaluation	31
6.8.1	Response Time	31
6.8.2	Concurrent Users	31
6.8.3	Resource Utilization	31
6.9	Requirement Verification	31
6.10	Results and Discussion	31
6.11	Limitations	31
6.12	Future Evaluation Opportunities	32
6.13	Chapter Summary	32
7	Conclusion and Future Work	33
7.1	Introduction	33
7.2	Project Summary	33
7.3	Objectives Achievement	34
7.4	Project Contributions	34
7.5	Project Limitations	35
7.6	Future Work	35
7.7	Lessons Learned	36
7.8	Final Remarks	36
7.9	Chapter Summary	36
A	Database Reference	38
B	REST API Reference	39
C	Additional User Interface Screens	40

D Design Decisions and Assumptions**41**

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
ETA	Estimated Time of Arrival
FCM	Firebase Cloud Messaging
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

Chapter 1

Introduction

1.1 Background

Emergency response systems play a critical role in protecting lives and property by coordinating the actions of emergency service providers such as ambulance services, fire departments, and law enforcement agencies. The effectiveness of these systems depends on their ability to receive incident reports, assess their urgency, allocate appropriate resources, and maintain continuous communication between responders throughout the incident lifecycle. As urban populations continue to grow and emergency situations become increasingly complex, traditional dispatch approaches face significant challenges in maintaining rapid response times and efficient resource utilization [1], [2].

Recent advances in software engineering, cloud computing, real-time communication technologies, and artificial intelligence have enabled the development of intelligent decision-support systems capable of assisting emergency operators during critical situations. These technologies can improve situational awareness, recommend resource allocation, estimate response times, and assist with operational decision-making while maintaining human oversight [3], [4].

Despite these technological advances, many educational projects and prototype systems focus only on basic incident reporting, overlooking the complexity of real emergency dispatch operations. Consequently, there is an opportunity to develop a comprehensive simulation platform that demonstrates the complete emergency dispatch workflow while remaining suitable for academic study.

1.2 Problem Statement

Efficient emergency response requires accurate incident reporting, rapid prioritization, coordinated communication among multiple emergency agencies, and continuous mon-

itoring of response units. Developing and evaluating such systems presents significant challenges because real emergency dispatch infrastructures are tightly integrated with governmental organizations, public safety agencies, and communication networks that are inaccessible for academic projects [1].

Many existing academic implementations simplify emergency management into a basic reporting application, where a citizen submits an emergency request and an administrator manually approves it. Such simplified workflows fail to represent the complexity of real dispatch operations, including incident prioritization, responder coordination, hospital selection, vehicle tracking, and real-time communication.

As a result, there is a need for an educational platform capable of realistically simulating emergency dispatch operations without requiring integration with national emergency services.

1.3 Proposed Solution

NAJDA is proposed as an intelligent emergency dispatch simulation platform designed specifically for educational and research purposes. Rather than replacing existing emergency infrastructures, NAJDA simulates the operational workflow of a modern emergency response center, allowing multiple user roles to collaborate within a realistic dispatch environment.

The platform supports seven primary actors:

- Citizen
- Dispatcher
- Ambulance Crew
- Firefighter
- Police Officer
- Hospital Staff
- Administrator

NAJDA employs an event-driven microservice architecture to coordinate communication between system components while integrating artificial intelligence modules that provide decision-support recommendations for incident prioritization, hospital selection, and hazard classification. Throughout the system, AI serves only as a recommendation engine, ensuring that all operational decisions remain under human control.

1.4 Project Objectives

The primary objective of NAJDA is to design and implement a realistic emergency dispatch simulation platform that demonstrates modern software engineering principles while supporting the coordination of multiple emergency response agencies.

The specific objectives of the project are as follows:

- Design a scalable event-driven microservice architecture.
- Simulate the complete emergency incident lifecycle.
- Provide dedicated interfaces for all supported user roles.
- Implement real-time incident monitoring and vehicle tracking.
- Integrate artificial intelligence modules for decision support.
- Demonstrate secure authentication and role-based authorization.
- Develop a platform suitable for academic demonstration and evaluation.

1.5 Project Scope

NAJDA is designed exclusively as a simulation platform for academic purposes. The project models the workflow of emergency dispatch centers while avoiding direct integration with governmental emergency communication infrastructures.

The project includes:

- Emergency incident reporting.
- Incident prioritization.
- Dispatcher-assisted unit assignment.
- Vehicle tracking.
- Hospital recommendation.
- Mission monitoring.
- Administrative management.
- Analytics and reporting.

The project explicitly excludes:

- Integration with national emergency telephone services.
- Real ambulance, police, or fire department communication systems.
- Medical diagnosis or treatment recommendations.
- Autonomous dispatch decisions without human approval.

1.6 Research Methodology Overview

The development of NAJDA follows an iterative software engineering approach based on Agile principles. The project is implemented incrementally, beginning with a functional minimum viable product, followed by the integration of artificial intelligence capabilities and finally system refinement and optimization [1].

The software architecture emphasizes modularity, scalability, maintainability, and loose coupling through the adoption of event-driven microservices. Each service is developed independently while communicating asynchronously through a message broker to improve system flexibility and fault isolation [4], [5].

A detailed discussion of the adopted software development lifecycle and project management methodology is presented in a later chapter.

1.7 Report Organization

This thesis is organized into seven chapters.

Chapter 2 presents the system requirements, stakeholders, functional requirements, non-functional requirements, and overall workflow analysis.

Chapter 3 reviews related literature and existing technologies relevant to emergency dispatch systems, artificial intelligence, and distributed software architectures.

Chapter 4 describes the complete system design, including the architectural model, database design, communication mechanisms, security considerations, and deployment architecture.

Chapter 5 explains the implementation of the proposed solution and discusses the technologies, frameworks, and development environment used throughout the project.

Chapter 6 presents the testing methodology, system evaluation, experimental observations, and identified limitations.

Finally, Chapter 7 summarizes the work presented in this thesis and discusses possible directions for future enhancement.

Chapter 2

Requirements Analysis

2.1 Introduction

Requirements analysis is a fundamental phase in software engineering that establishes the functional and non-functional expectations of a system before design and implementation begin. A clear understanding of stakeholder needs ensures that the proposed solution satisfies its intended objectives while providing a foundation for architectural decisions [1], [2].

This chapter identifies the stakeholders of the NAJDA platform, defines the system requirements, describes the primary use cases, and presents the overall operational workflow that the system is expected to support.

2.2 Stakeholders

NAJDA is designed to support multiple categories of users, each with distinct responsibilities throughout the emergency response process.

- **Citizens** report emergencies, provide incident details, upload supporting media, monitor incident status, and receive updates.
- **Dispatchers** evaluate reported incidents, assign emergency units, monitor ongoing missions, and coordinate emergency operations.
- **Ambulance Crews** receive assigned medical missions, update mission status, navigate to incident locations, and transport patients to hospitals when necessary.
- **Firefighters** respond to fire-related incidents, provide operational updates, and report mission completion.

- **Police Officers** respond to security and law enforcement incidents while maintaining communication with dispatchers.
- **Hospital Staff** manage hospital availability, monitor incoming patients, and update hospital capacity information.
- **Administrators** manage users, emergency vehicles, hospitals, system configuration, and operational statistics.

2.2.1 Stakeholder Responsibility Matrix

Placeholder: Table describing each stakeholder, responsibilities, permissions, and primary interactions with the system.

2.3 System Actors

NAJDA supports seven primary actors:

1. Citizen
2. Dispatcher
3. Ambulance Crew
4. Firefighter
5. Police Officer
6. Hospital Staff
7. Administrator

Each actor interacts with the system according to permissions assigned to their role, ensuring operational security and responsibility separation.

2.4 Functional Requirements

The NAJDA platform shall provide the following functional capabilities.

2.4.1 Citizen Functions

...

2.4.2 Dispatcher Functions

...

2.4.3 Responder Functions

...

2.4.4 Hospital Functions

...

2.4.5 Administrator Functions

...

2.4.6 Functional Requirements Matrix

Placeholder: Software Requirements Specification (SRS) table containing requirement ID, description, actor, and priority.

2.5 Non-Functional Requirements

2.5.1 Performance

...

2.5.2 Scalability

...

2.5.3 Availability

...

2.5.4 Maintainability

...

2.5.5 Usability

...

2.5.6 Security

...

2.5.7 Non-Functional Requirements Matrix

Placeholder: Table summarizing measurable quality attributes including performance, scalability, security, reliability, maintainability, and usability.

2.6 User Stories

Placeholder: User stories for each actor following the format "As a..., I want..., so that...".

2.7 Use Case Analysis

The overall operation of NAJDA begins when a citizen submits an emergency report through the mobile application. After the incident is created, the dispatcher evaluates the reported information and coordinates the appropriate emergency response. Assigned responders receive mission notifications, update mission progress throughout the response process, and complete the mission once the incident has been resolved. Hospital staff participate when patient transportation is required, while administrators supervise overall system management.

2.7.1 Overall Use Case Diagram

Placeholder: UML Use Case Diagram.

2.7.2 Use Case Specifications

Placeholder: Detailed specifications for the primary use cases including preconditions, postconditions, normal flow, and alternative flows.

2.8 Business Rules

NAJDA follows several operational rules that govern system behavior.

- Every emergency incident shall have exactly one dispatcher responsible for coordination.

- Emergency vehicles shall only receive missions when an active crew has been assigned to the vehicle.
- Artificial intelligence modules provide recommendations only; all operational decisions remain under human control.
- Users shall only access functions permitted by their assigned role.
- Incident cancellation shall follow predefined operational constraints depending on the current mission status.
- Every incident shall maintain a complete history of operational events.

2.9 Assumptions and Constraints

The following assumptions apply throughout the project.

2.9.1 *Assumptions*

- The platform operates as an academic simulation.
- Mobile devices are assumed to have GPS connectivity.
- Internet connectivity is available during normal operation.
- Emergency organizations participating in the simulation maintain updated operational information.
- Hospital availability information is assumed to be accurate.

2.9.2 *Constraints*

- The project does not integrate with governmental emergency communication systems.

2.10 Overall System Workflow

The overall workflow begins when a citizen reports an emergency through the mobile application. The system records the incident and provides decision-support recommendations to assist dispatchers in evaluating its priority. Based on the available information, the dispatcher assigns suitable emergency units to respond.

Assigned responders receive mission notifications, travel to the incident location, continuously update mission progress, and complete the assigned task. When medical

transportation is required, hospital staff receive advance notification before patient arrival. Throughout the incident lifecycle, the system records operational events for monitoring, reporting, and future analysis.

2.10.1 Workflow Activity Diagram

Placeholder: UML Activity Diagram illustrating the complete emergency response workflow.

2.11 Requirements Traceability Matrix

Placeholder: Traceability matrix linking functional requirements to use cases, design components, and test cases.

2.12 Chapter Summary

This chapter identified the stakeholders of NAJDA and defined the functional and non-functional requirements that guide the development of the system. It also established the primary business rules, operational assumptions, and overall workflow that form the basis for the architectural design presented in the following chapters.

Chapter 3

Literature Review

3.1 Introduction

3.2 Introduction

Emergency response systems have evolved significantly with the advancement of communication technologies, cloud computing, and artificial intelligence. Modern emergency management platforms aim to improve response time, enhance coordination between multiple agencies, and support decision-making through real-time information sharing [1], [3].

This chapter reviews the concepts, technologies, and research relevant to the development of NAJDA. It also identifies limitations in existing approaches and highlights the research gap addressed by the proposed system.

3.3 Emergency Dispatch Systems

Emergency dispatch systems are responsible for receiving incident reports, assessing their severity, allocating emergency resources, and coordinating response teams throughout the incident lifecycle. Their primary objective is to minimize response time while ensuring that available resources are utilized efficiently.

Traditional Computer-Aided Dispatch (CAD) systems are widely used by emergency organizations such as ambulance services, police departments, and fire brigades. These systems assist dispatchers in recording incidents, monitoring responder locations, assigning vehicles, and maintaining communication with field personnel. Modern dispatch systems further incorporate geographic information systems (GIS), digital mapping, and real-time tracking technologies to improve operational awareness [1], [2].

Despite their capabilities, most production dispatch systems are designed for governmental agencies and are tightly integrated with national emergency communication infrastructures, making them unsuitable for educational implementation or academic experimentation.

3.4 Artificial Intelligence in Emergency Response

Artificial intelligence has become an increasingly valuable tool in emergency management by assisting operators in making informed decisions under time constraints. AI techniques have been applied to incident classification, emergency prioritization, traffic prediction, image analysis, hospital recommendation, and resource allocation [6], [7].

Machine learning models can analyze historical incident data to estimate incident severity or predict emergency demand in different geographical regions. Computer vision models can assist in identifying hazards from uploaded images, while optimization algorithms can recommend hospitals based on travel distance, capacity, and operational availability.

However, ethical considerations require that AI systems remain decision-support tools rather than autonomous decision-makers. Consequently, many modern emergency management systems adopt a human-in-the-loop approach, where artificial intelligence generates recommendations while trained operators retain full authority over final decisions. NAJDA follows this principle throughout its architecture.

3.5 Microservice Architecture

Microservice architecture has become a popular software design approach for large-scale distributed applications. Unlike monolithic systems, microservices divide application functionality into independent services that communicate through lightweight protocols or asynchronous messaging [4], [5].

Each microservice is responsible for a specific business capability and can be developed, deployed, and maintained independently. This separation improves scalability, maintainability, fault isolation, and technology flexibility while enabling development teams to work concurrently on different services [3], [4].

For applications requiring real-time communication between multiple independent components, event-driven microservices provide additional advantages by reducing service coupling and supporting asynchronous processing [5].

The NAJDA platform adopts an event-driven microservice architecture to support modular development and efficient coordination between emergency management ser-

vices.

3.6 Event-Driven Systems

Event-driven architecture is a software design paradigm in which system components communicate by publishing and consuming events rather than invoking each other directly. This approach improves scalability and reduces dependencies between services [5].

Message brokers such as RabbitMQ enable asynchronous communication between distributed services while ensuring reliable message delivery and supporting fault tolerance [8]. In emergency response systems, event-driven communication allows different system components to react independently to incident updates, mission assignments, responder status changes, and hospital availability updates without introducing unnecessary coupling.

NAJDA utilizes RabbitMQ as the primary event broker for inter-service communication while employing WebSocket technology to provide real-time updates to connected users.

3.7 Real-Time Communication Technologies

Emergency management applications require continuous synchronization between dispatch centers and field responders. Technologies such as WebSockets provide persistent bidirectional communication channels that enable instant transmission of incident updates, responder locations, and operational status changes **websocket**.

Compared with traditional polling techniques, WebSockets significantly reduce communication latency and network overhead, making them suitable for applications that require live monitoring and immediate notification of critical events.

Push notification services further enhance communication by delivering mission assignments and important alerts directly to mobile devices, ensuring that responders receive updates even when applications are operating in the background. NAJDA employs Firebase Cloud Messaging to implement this functionality [9].

3.8 Cloud Services and Backend Technologies

Cloud platforms provide scalable infrastructure for authentication, data storage, notification delivery, and database management. Backend services commonly rely on relational database management systems for transactional consistency while utilizing in-memory databases for caching frequently accessed information [10], [11].

Modern software platforms also integrate cloud authentication services to simplify identity management while improving security through features such as phone verification and multi-factor authentication [9].

NAJDA combines PostgreSQL as its primary database, Redis for temporary live-location caching, and Firebase services for authentication, cloud messaging, and media storage to achieve a balance between scalability, performance, and implementation simplicity.

3.9 Related Work

Numerous research projects and commercial applications have explored emergency response management from different perspectives. Some systems emphasize geographic information systems and route optimization, while others focus on intelligent incident classification, hospital recommendation, or predictive resource allocation.

Although these systems provide valuable functionality, many are designed for operational deployment within governmental organizations or address only a limited portion of the emergency response workflow. Academic projects frequently simplify emergency management to basic incident reporting applications without modeling realistic dispatch operations or coordination among multiple emergency services [1], [2].

NAJDA differs by simulating the complete operational workflow of a dispatch center while integrating modern software engineering practices, event-driven microservices, real-time communication, and AI-assisted decision support within a single educational platform.

3.10 Research Gap

Existing emergency dispatch solutions generally target operational environments and therefore require specialized infrastructure, governmental integration, and restricted communication networks. Such dependencies make them unsuitable for educational use or independent academic development.

On the other hand, many academic implementations oversimplify emergency management workflows and do not adequately represent dispatcher decision-making, responder coordination, shift management, or event-driven communication [4], [5].

NAJDA addresses this gap by providing a realistic emergency dispatch simulation platform that demonstrates modern software architecture, artificial intelligence integration, and multi-role coordination while remaining independent of real-world emergency communication systems.

3.11 Chapter Summary

This chapter reviewed the technologies and research areas that form the foundation of NAJDA, including emergency dispatch systems, artificial intelligence, microservice architecture, event-driven communication, real-time technologies, and cloud services. The identified research gap demonstrates the need for an educational platform capable of realistically simulating emergency dispatch operations while showcasing modern software engineering practices. The following chapter presents the system requirements and analysis derived from these observations.

Chapter 4

System Design

4.1 Introduction

System design transforms the requirements identified in the previous chapters into a detailed technical architecture that can be implemented efficiently while remaining scalable, maintainable, and extensible. This chapter presents the architectural design of NAJDA, including its overall system architecture, software components, database structure, communication mechanisms, security model, and artificial intelligence integration.

The proposed design follows an event-driven microservice architecture in which independent services communicate asynchronously through an event broker while providing real-time updates to connected clients. The architecture emphasizes modularity, fault isolation, scalability, and clear separation of responsibilities [3], [4], [5].

4.2 Design Goals

The NAJDA architecture was designed to achieve the following goals:

- Scalability
- Maintainability
- Loose coupling
- High cohesion
- Fault isolation
- Real-time communication
- Security

- Extensibility

4.3 Overall System Architecture

This section presents the high-level architecture of NAJDA and illustrates how the major software components interact to support emergency dispatch operations.

4.3.1 Overall Architecture Diagram

Placeholder: High-Level System Architecture Diagram

4.4 Architectural Views

4.4.1 System Context View

Placeholder: Context Diagram

4.4.2 Container View

Placeholder: C4 Container Diagram

4.4.3 Component View

Placeholder: Component Diagram

4.5 Microservice Architecture

NAJDA follows an event-driven microservice architecture in which each service is responsible for a single business capability. Services communicate asynchronously through an event broker, reducing direct dependencies and improving scalability. This architectural style promotes independent deployment, loose coupling, and improved maintainability when compared with traditional monolithic systems [3], [4], [5].

4.5.1 Service Responsibilities

Placeholder: Table describing each microservice and its responsibilities.

4.5.2 Inter-Service Communication

Placeholder: RabbitMQ Communication Diagram

4.6 Database Design

Persistent system data is stored within a relational database designed to maintain data consistency while supporting efficient querying and future extensibility. PostgreSQL serves as the system of record due to its transactional reliability and support for complex relational data, while Redis is utilized for temporary caching of rapidly changing location information to improve application responsiveness [10], [11].

4.6.1 *Entity Relationship Diagram*

Placeholder: ER Diagram

4.6.2 *Database Schema*

Placeholder: Database Schema Description

4.7 Class Design

The object-oriented design of the system is represented using UML class diagrams illustrating the relationships between the principal domain entities.

4.7.1 *Class Diagram*

Placeholder: UML Class Diagram

4.8 System Behaviour

This section describes how different system components interact throughout the execution of important business processes.

4.8.1 *Incident Reporting Sequence*

Placeholder: Sequence Diagram

4.8.2 *Mission Assignment Sequence*

Placeholder: Sequence Diagram

4.8.3 *Hospital Assignment Sequence*

Placeholder: Sequence Diagram

4.9 Incident State Model

Emergency incidents transition through multiple operational states during their life-cycle.

4.9.1 State Diagram

Placeholder: Incident State Diagram

4.10 Artificial Intelligence Design

Artificial intelligence within NAJDA assists dispatchers by generating recommendations while preserving human decision-making authority. The design follows a human-in-the-loop approach in which AI provides decision support while final operational decisions remain under human control [6], [7].

The AI subsystem provides:

- Incident priority prediction.
- Duplicate incident detection.
- Hospital recommendation.
- Hazard image classification.

4.10.1 AI Architecture

Placeholder: AI Architecture Diagram

4.11 Authentication and Authorization

NAJDA employs role-based access control (RBAC) to ensure that users only access functionality appropriate to their assigned responsibilities. Authentication is implemented using Firebase Authentication, while authorization is enforced within backend services according to the authenticated user's assigned role [9].

4.11.1 Authentication Flow

Placeholder: Authentication Flow Diagram

4.11.2 Authorization Model

Placeholder: RBAC Diagram

4.12 Real-Time Communication

NAJDA supports real-time communication between users and backend services to ensure that emergency information is synchronized across all participating actors. RabbitMQ is used for asynchronous communication between backend services, while WebSockets deliver live updates to connected clients with minimal latency **websocket**, [8].

4.12.1 Real-Time Event Flow

Placeholder: Event Flow Diagram

4.13 Deployment Architecture

The deployment architecture describes how client applications, backend services, databases, cloud services, and supporting infrastructure are organized within the production environment. The distributed deployment model follows cloud-native design principles, enabling independent deployment of services while utilizing managed cloud services for authentication, storage, notifications, and persistent data management [9], [10].

4.13.1 Deployment Diagram

Placeholder: UML Deployment Diagram

4.14 Security Considerations

The security design of NAJDA focuses on protecting user identities, restricting access to authorized functionality, and ensuring secure communication between distributed components. The system combines secure authentication, role-based authorization, encrypted communication channels, and server-side access validation to protect sensitive operational data [1], [9].

4.14.1 Security Architecture

Placeholder: Security Architecture Diagram

4.15 Design Decisions

The following major architectural decisions influenced the development of NAJDA:

- Adoption of an event-driven microservice architecture.
- Separation of AI functionality into an independent service.
- Role-based access control.
- Event-driven communication between services.
- Dedicated relational database for persistent data.
- In-memory caching for frequently changing live data.
- Cloud-based authentication and notification services.
- Human-in-the-loop AI decision support.

4.16 Chapter Summary

This chapter presented the complete architectural design of NAJDA, including its software architecture, component organization, database design, communication mechanisms, artificial intelligence subsystem, security model, and deployment strategy. These design decisions provide the foundation for the implementation discussed in the following chapter.

Chapter 5

Implementation

5.1 Introduction

This chapter describes the implementation of the NAJDA platform based on the architectural design presented in the previous chapter. It discusses the technologies, frameworks, development environment, and implementation approach adopted for each major subsystem. The chapter also highlights how the different software components interact to provide a complete emergency dispatch simulation platform.

The implementation follows the event-driven microservice architecture introduced in the previous chapter, combining modern backend frameworks, cloud services, real-time communication technologies, and artificial intelligence components to build a scalable and maintainable emergency dispatch simulation platform **flutter**, [12], [13], [14].

5.2 Development Environment

The NAJDA platform was developed using modern software engineering tools and frameworks that support modular development, scalability, maintainability, and rapid application development. Open-source frameworks and managed cloud services were selected to simplify implementation while ensuring compatibility with modern software engineering practices **flutter**, [12], [13], [14].

5.2.1 Software Tools

Placeholder: Table listing the software development tools and versions used during implementation.

5.2.2 *Technology Stack*

Placeholder: Technology Stack Table

5.3 Backend Implementation

The backend consists of multiple independent services responsible for handling business logic, data management, authentication, incident processing, dispatch operations, and communication between system components. Each service is implemented using Spring Boot and communicates asynchronously through RabbitMQ following an event-driven architecture [8], [12].

5.3.1 *Microservice Implementation*

Placeholder: Description of each implemented service.

5.3.2 *REST API Implementation*

Placeholder: API endpoint examples and controller implementation.

5.3.3 *Event Processing*

Placeholder: RabbitMQ publishers, consumers, and event handlers.

5.4 Database Implementation

The relational database stores all persistent operational data while maintaining consistency between the different software modules. PostgreSQL serves as the primary system of record, while Redis is used to cache frequently changing live vehicle location data to improve system responsiveness [10], [11].

5.4.1 *Database Tables*

Placeholder: Important database tables.

5.4.2 *Relationships*

Placeholder: Example database relationships.

5.4.3 *Migration Strategy*

Placeholder: Database migration approach.

5.5 Mobile Application Implementation

The mobile application provides dedicated interfaces for citizens and emergency responders. It is developed using Flutter to support cross-platform deployment from a single codebase while maintaining consistent user experience across Android and iOS devices **flutter**.

5.5.1 *Application Structure*

Placeholder: Mobile project structure.

5.5.2 *Citizen Features*

Placeholder: Screenshots and implementation details.

5.5.3 *Responder Features*

Placeholder: Screenshots and implementation details.

5.6 Web Application Implementation

The web application provides interfaces for dispatchers, hospital staff, and administrators. It is implemented using Next.js, enabling server-side rendering, efficient routing, and modern React-based development practices [13].

5.6.1 *Dispatcher Dashboard*

Placeholder: Dashboard screenshots and explanation.

5.6.2 *Hospital Dashboard*

Placeholder: Screenshots and explanation.

5.6.3 *Administration Dashboard*

Placeholder: Screenshots and explanation.

5.7 Artificial Intelligence Implementation

The AI subsystem provides intelligent recommendations that assist dispatchers while maintaining human control over operational decisions. Separate machine learning models are used for priority prediction and hazard classification, while deterministic algorithms are employed where explainability and predictable behaviour are preferred [6], [7].

5.7.1 *Priority Prediction*

Placeholder: XGBoost implementation.

5.7.2 *Duplicate Detection*

Placeholder: Rule-based duplicate detection implementation.

5.7.3 *Hospital Recommendation*

Placeholder: Weighted scoring algorithm.

5.7.4 *Hazard Classification*

Placeholder: MobileNetV2 implementation.

5.8 Cloud Services Integration

NAJDA integrates several Firebase cloud services to simplify authentication, notification delivery, and media storage. Firebase provides managed cloud infrastructure that reduces implementation complexity while supporting secure authentication and reliable mobile communication [9].

5.8.1 *Authentication*

Placeholder: Firebase Authentication implementation.

5.8.2 *Push Notifications*

Placeholder: Firebase Cloud Messaging implementation.

5.8.3 *File Storage*

Placeholder: Firebase Storage implementation.

5.9 Caching Implementation

Redis is employed as an in-memory cache for storing live GPS locations of emergency vehicles. Using Redis reduces database load and enables fast retrieval of frequently changing location information required for dispatcher dashboards and responder tracking [11].

Placeholder: Redis implementation for live GPS data.

5.10 Security Implementation

NAJDA employs multiple security mechanisms to protect user identities, restrict access to authorized functionality, and secure communication between client applications and backend services. Authentication is handled using Firebase Authentication, while authorization is enforced through role-based access control implemented within the backend services [9].

5.10.1 Authentication

Placeholder: Firebase Authentication implementation using Phone OTP, Firebase ID Tokens, and backend token verification through the Firebase Admin SDK.

5.10.2 Authorization

Placeholder: Role-Based Access Control (RBAC) implementation using authenticated Firebase users and application-specific roles stored in PostgreSQL.

5.10.3 Secure Communication

Placeholder: HTTPS communication, authenticated API requests, and secure Web-Socket connections.

5.11 Deployment

The NAJDA platform is designed for deployment using managed cloud services and independently deployable backend components. This deployment approach supports scalability, simplified maintenance, and independent service updates while leveraging managed infrastructure for authentication, notifications, and data storage [9], [10].

5.11.1 Production Environment

Placeholder: Hosting architecture.

5.11.2 Deployment Pipeline

Placeholder: Deployment process.

5.12 Challenges Encountered

Placeholder: Technical challenges faced during implementation and how they were resolved.

5.13 Chapter Summary

This chapter presented the implementation of the NAJDA platform, including the backend services, mobile and web applications, database, artificial intelligence subsystem, cloud services, security mechanisms, and deployment strategy. The following chapter evaluates the implemented system through testing and performance analysis.

Chapter 6

Testing and Evaluation

6.1 Introduction

Testing and evaluation are essential phases of software engineering that verify whether a system satisfies its specified requirements while maintaining reliability, correctness, and acceptable performance. Software testing aims to detect defects, validate system behavior, and provide confidence that the implemented software satisfies its intended objectives before deployment [1], [2].

This chapter presents the testing strategy adopted for NAJDA, describes the different levels of testing performed, evaluates the system against its functional and non-functional requirements, and discusses the obtained results.

6.2 Testing Strategy

NAJDA was evaluated using multiple levels of software testing to ensure that individual components function correctly and that the complete system operates as expected when integrated. The adopted testing strategy follows widely accepted software engineering practices by combining component-level verification with end-to-end validation of system functionality [1], [15].

The testing strategy consisted of:

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing
- Performance Evaluation

6.3 Testing Environment

6.3.1 *Hardware Environment*

Placeholder: Hardware specifications used during testing.

6.3.2 *Software Environment*

Placeholder: Software versions, operating systems, databases, browsers, and development tools.

6.4 Unit Testing

Unit testing was performed to verify the correctness of individual software components in isolation. Each software module was tested independently before integration to ensure that its functionality satisfied the intended design [16].

6.4.1 *Backend Unit Tests*

Placeholder: Service-layer and controller test results.

6.4.2 *Frontend Unit Tests*

Placeholder: UI component tests.

6.4.3 *AI Module Tests*

Placeholder: AI model verification tests.

6.5 Integration Testing

Integration testing verified communication between the different system modules. This phase focused on validating interactions between backend services, databases, cloud services, and asynchronous messaging components after successful unit testing [1].

6.5.1 *Backend Integration*

Placeholder: Microservice communication tests.

6.5.2 *Event Processing*

Placeholder: RabbitMQ event verification.

6.5.3 *Cloud Service Integration*

Placeholder: Firebase integration tests.

6.6 System Testing

System testing evaluated the complete NAJDA platform under realistic operational scenarios. End-to-end workflows were executed to verify that all software components operated correctly as an integrated system [2].

6.6.1 *Emergency Reporting*

Placeholder: Test scenario and results.

6.6.2 *Mission Assignment*

Placeholder: Test scenario and results.

6.6.3 *Hospital Recommendation*

Placeholder: Test scenario and results.

6.6.4 *Incident Completion*

Placeholder: Test scenario and results.

6.7 User Acceptance Testing

User Acceptance Testing (UAT) evaluates whether a software system satisfies user expectations and operational requirements before deployment. During this phase, representative users execute predefined scenarios and provide feedback regarding usability, functionality, and overall system behavior [15].

Placeholder: User Acceptance Testing methodology and participant feedback.

6.8 Performance Evaluation

Performance testing evaluated the responsiveness and scalability of the implemented system. The objective was to measure the system’s ability to process requests efficiently while maintaining acceptable response times under varying workloads [1].

6.8.1 Response Time

Placeholder: Response time measurements.

6.8.2 Concurrent Users

Placeholder: Concurrent user testing results.

6.8.3 Resource Utilization

Placeholder: CPU, memory, and database performance measurements.

6.9 Requirement Verification

Placeholder: Table mapping functional requirements to completed test cases and outcomes.

6.10 Results and Discussion

This section analyzes the testing outcomes and evaluates whether the implemented system satisfies the objectives established in the requirements analysis.

Placeholder: Discussion of testing results and observed system behavior.

6.11 Limitations

Although NAJDA successfully demonstrates a realistic emergency dispatch simulation platform, several limitations remain.

- The platform operates as an academic simulation and is not connected to governmental emergency services.
- AI recommendations are intended for demonstration purposes and should not replace professional decision-making.
- Performance evaluation is limited to the available testing infrastructure.

- Real-world emergency datasets were not available for large-scale validation.

6.12 Future Evaluation Opportunities

Future work may include:

- Large-scale load testing.
- Formal usability studies.
- AI model retraining using larger datasets.
- Realistic simulation with multiple simultaneous incidents.

6.13 Chapter Summary

This chapter presented the testing methodology adopted for NAJDA and evaluated the implemented system through unit, integration, system, and performance testing. The obtained results demonstrate the correctness of the implemented functionality while identifying limitations and opportunities for future improvements.

Chapter 7

Conclusion and Future Work

7.1 Introduction

This chapter concludes the NAJDA graduation project by summarizing the work presented throughout this thesis. It reviews the project objectives, highlights the major achievements, discusses the limitations of the current implementation, and outlines potential directions for future enhancement.

The conclusions presented in this chapter are based on the design, implementation, and evaluation activities described throughout the previous chapters and follow accepted software engineering practices for project assessment and continuous improvement [1], [2].

7.2 Project Summary

NAJDA was developed as an emergency dispatch simulation platform intended for academic and educational purposes. The project demonstrates how modern software engineering practices can be applied to simulate emergency response operations without requiring integration with governmental emergency infrastructures.

The platform supports multiple user roles, including citizens, dispatchers, ambulance crews, firefighters, police officers, hospital staff, and system administrators. By combining real-time communication, distributed software architecture, and artificial intelligence-assisted decision support, NAJDA provides a realistic representation of emergency dispatch workflows while maintaining human control over operational decisions. The adopted architecture demonstrates how modern software engineering principles can be integrated to produce scalable and maintainable distributed applications [3], [4].

7.3 Objectives Achievement

The primary objectives established at the beginning of the project have been addressed through the design and implementation of the proposed platform.

The project successfully demonstrates:

- A realistic emergency dispatch simulation workflow.
- Multi-role coordination between emergency participants.
- Event-driven communication between distributed software components.
- AI-assisted decision support while preserving human authority.
- Real-time incident monitoring and mission management.
- A scalable and modular software architecture suitable for future expansion.

Placeholder: Final evaluation of objectives after project completion.

7.4 Project Contributions

The NAJDA platform contributes to software engineering education by providing a comprehensive case study that combines multiple modern technologies within a single integrated application.

The main contributions include:

- Design of an event-driven microservice architecture for emergency dispatch simulation.
- Integration of multiple emergency service roles within a unified platform.
- Application of artificial intelligence as a decision-support mechanism rather than an autonomous decision-maker.
- Demonstration of real-time communication techniques for mission coordination.
- Development of an extensible platform that can support future academic research and educational activities.

7.5 Project Limitations

Although NAJDA demonstrates the core concepts of a modern emergency dispatch platform, several limitations remain.

- The platform operates solely as an academic simulation.
- Integration with governmental emergency communication infrastructure is intentionally excluded.
- Artificial intelligence models rely on limited datasets suitable for educational purposes.
- Performance evaluation depends on the available development and testing environment.
- Certain advanced routing and traffic prediction capabilities remain outside the scope of the current implementation.

7.6 Future Work

Several improvements may be incorporated in future versions of NAJDA to enhance both realism and functionality.

Potential future work includes:

- Integration with real-time traffic and routing services.
- Advanced geographic information system (GIS) capabilities.
- Improved emergency demand prediction using larger datasets.
- Enhanced AI models trained on real emergency response data.
- Offline communication mechanisms for unstable network environments.
- Multi-region deployment with distributed infrastructure.
- Expanded analytics and operational reporting dashboards.
- Wearable device integration for emergency responders.
- Tablet-optimized dispatcher interfaces.
- Comprehensive usability studies involving emergency management professionals.

7.7 Lessons Learned

The development of NAJDA provided valuable practical experience in software engineering, distributed systems, artificial intelligence, cloud computing, mobile application development, and system integration. The project also demonstrated the importance of requirements engineering, modular architecture, effective communication between software components, and iterative development throughout the software development lifecycle.

Placeholder: Add team-specific lessons learned after project completion.

7.8 Final Remarks

NAJDA demonstrates that a realistic emergency dispatch platform can be developed as an educational simulation while incorporating modern software engineering principles, distributed system architecture, and artificial intelligence-assisted decision support. The project serves as both a practical software engineering solution and an academic case study illustrating the design and implementation of complex real-time information systems.

It is anticipated that the concepts, architecture, and implementation presented in this thesis will provide a solid foundation for future research, educational projects, and further development in intelligent emergency management systems.

7.9 Chapter Summary

This chapter concluded the NAJDA project by summarizing the work presented throughout the thesis, evaluating the achievement of the project objectives, discussing current limitations, and identifying opportunities for future enhancement. Together with the previous chapters, it completes the documentation of the proposed emergency dispatch simulation platform.

Bibliography

- [1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill, 2020.
- [2] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [3] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2022.
- [4] S. Newman, *Building Microservices*, 2nd ed. O'Reilly Media, 2021.
- [5] C. Richardson, *Microservices Patterns*. Manning Publications, 2018.
- [6] XGBoost Developers, *Xgboost documentation*, <https://xgboost.readthedocs.io>, 2025.
- [7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- [8] RabbitMQ, *Rabbitmq documentation*, <https://www.rabbitmq.com/docs>, 2025.
- [9] Google, *Firebase documentation*, <https://firebase.google.com/docs>, 2025.
- [10] PostgreSQL Global Development Group, *Postgresql documentation*, <https://www.postgresql.org/docs/>, 2025.
- [11] Redis, *Redis documentation*, <https://redis.io/docs>, 2025.
- [12] VMware, *Spring boot documentation*, <https://spring.io/projects/spring-boot>, 2025.
- [13] Vercel, *Next.js documentation*, <https://nextjs.org/docs>, 2025.
- [14] S. Ramírez, *Fastapi documentation*, <https://fastapi.tiangolo.com>, 2025.
- [15] International Software Testing Qualifications Board, *Istqb certified tester foundation level syllabus version 4.0*, 2023. [Online]. Available: <https://www.istqb.org/>.
- [16] JUnit Team, *JUnit 5 user guide*, 2025. [Online]. Available: <https://junit.org/junit5/docs/current/user-guide/>.

Appendix A

Database Reference

This appendix contains supplementary database documentation for the NAJDA platform.

It will include:

- Complete database schema.
- Entity descriptions.
- Table attributes.
- Primary and foreign key relationships.
- Data types and constraints.
- Database normalization notes.

The appendix serves as a technical reference and complements the database design discussed in Chapter 4.

Appendix B

REST API Reference

This appendix provides the REST API reference for the NAJDA backend services.

The final version will include:

- Authentication endpoints.
- Incident management endpoints.
- Dispatch endpoints.
- Hospital management endpoints.
- Administrative endpoints.
- Example request and response payloads.
- HTTP status codes.

Detailed endpoint implementation is discussed throughout Chapter 5.

Appendix C

Additional User Interface Screens

This appendix contains supplementary user interface designs that are not included within the main chapters.

Examples include:

- Citizen mobile application screens.
- Dispatcher dashboard screens.
- Ambulance crew interface.
- Firefighter interface.
- Police interface.
- Hospital staff interface.
- Administrator dashboard.

Only representative interfaces are discussed in the implementation chapter, while the remaining designs are provided here for completeness.

Appendix D

Design Decisions and Assumptions

This appendix summarizes the major architectural decisions made during the design of RapidAid together with the assumptions adopted throughout the project.

Topics documented include:

- Event-driven microservice architecture.
- Spring Boot backend services.
- FastAPI AI service.
- RabbitMQ event broker.
- PostgreSQL as the system of record.
- Redis for live GPS caching.
- Firebase Authentication, Cloud Messaging, and Storage.
- ShiftAssignment-based dispatch model.
- AI-assisted decision support philosophy.
- Simulation scope and project limitations.

The rationale for each decision is discussed in Chapter 4, while this appendix serves as a consolidated reference for future maintenance and project extension.